

Supply-Chain Disruption Triage with Responsible Multi-Agent AI

Shivendra | Track A - Agentic / Multi-Agent AI

1. Problem and domain

I chose S1: Supply Chain Risk and Optimization. A supplier delay is rarely an isolated event: its significance depends on current inventory, demand uncertainty, alternate stock, and the cost of acting too soon or too late. This makes it a meaningful AI opportunity, but also a domain where careless automation can create serious financial and operational loss. A system that expedites every late shipment wastes money; one that overlooks a genuine shortage can stop production.

My goal is therefore not to automate purchasing. It is to shorten the path from a disruption signal to a justified, reviewable recommendation. Agentic AI is useful here because the work is a sequence of bounded tasks: retrieve evidence, estimate impact, apply operating policy, explain the trade-off, and escalate. Well-designed agents can eventually take on longer workflows with less manual effort, but in a sensitive supply-chain setting that autonomy must be earned through controls, not assumed.

2. Approach and algorithm decisions

I used LangGraph to orchestrate three agents with typed messages and an explicit state graph. The Risk Detection and Evidence Agent queries shipment, purchase-order, and inventory data. The Demand and Impact Agent independently retrieves recent demand and estimates days of cover. The Replenishment Recommendation Agent produces only a bounded, reversible recommendation. LangGraph then pauses before human review; no simulated action proceeds without approval.

I chose LangGraph over CrewAI and AutoGen because this problem needs visible state transitions, durable checkpoints, conditional routing, and a hard human-in-the-loop gate. CrewAI is attractive for quick role-based prototypes, but its conversational abstraction is less precise for approval controls. AutoGen supports flexible agent dialogue, but that flexibility is not a benefit when a workflow must fail safely. The trade-off is more graph code in exchange for predictable behavior and an auditable trace.

I also rejected a single LLM prompt. It would mix fact retrieval, quantitative reasoning, and policy judgment in one opaque step. Instead, agents use allow-listed, parameterized, read-only DuckDB operations; the demand estimate is a transparent 28-day mean with a standard-deviation uncertainty signal. I considered a more complex forecasting model, but rejected it for this prototype because synthetic data cannot justify claims of superior real-world forecast accuracy. The simple baseline is easy for a planner to challenge and replace once historical data is available.

3. Results and error analysis

The evaluation is intentionally scenario-based rather than a misleading accuracy benchmark. The repository contains two reproducible end-to-end tests, both passing. In the success path, a verified five-day supplier delay for SKU-CRITICAL at Pune-DC produced an estimated 1.9 days of cover. The system generated a 280-unit inter-DC transfer proposal, stopped for human approval, and recorded the final simulated action. In the failure path, duplicate shipment updates with conflicting statuses and unverified provenance produced an escalation: 0 action proposals and 0 simulated executions. These are workflow safety metrics, not a claim of demand-forecast accuracy on a held-out real dataset.

The most important failure mode is silent confidence in bad data. The design fails closed on missing, unverified, or contradictory shipment evidence. It can still fail if verified source data is stale, if demand changes suddenly, or if alternate inventory is reserved elsewhere. Those risks are surfaced as assumptions in the trace and require planner review rather than being hidden by a fluent explanation.

4. Production and limitations

In production, I would separate read-only analytics from any ERP write path, use least-privilege credentials, and retain durable workflow checkpoints. Monitoring would cover data freshness, failed tool calls, approval and override rates, latency, cost, and recommendation outcomes. New policy or model versions would be replayed against historical cases and released through a canary, so the system can be updated without downtime.

The central limitation is the synthetic dataset. It proves that the workflow is reproducible and that safety controls behave as intended; it does not prove real-world business impact. Before deployment, I would run the system in shadow mode on historical ERP outcomes, calibrate thresholds with supply-chain planners, validate forecast performance by SKU and site, and only then widen the scope of recommendations.